Flask Websites: Quick Reference

Serving HTML, buttons with GET requests, forms with POST requests, and switching views

1. Project layout

Add two folders next to your API file. Flask finds them automatically by these exact names.

```
my_api/
|-- app.py            # your Flask API
|-- templates/        # HTML files  (index.html, view2.html)
`-- static/           # CSS, JavaScript, images  (app.js)
```

2. Serve a page

Put `index.html` in `templates/`, then add a route that renders it. Run the API and open `http://your_pi_ip:5000` in the browser.

```
from flask import Flask, render_template

@app.route('/')
def home():
    return render_template('index.html')   # looks inside templates/
```

Every HTML page starts from this skeleton:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Flask App</title>
</head>
<body>
  <h1>Welcome!</h1>        <!-- page content goes here -->
</body>
</html>
```

3. HTML tags cheat sheet

Get everything on the screen first, and use placeholders like [BUTTON HERE] for anything not listed. Styling comes later.

Element	HTML	Purpose
Headings	`<h1>Title</h1> ... <h6>`	h1 = main title, h2 = section, h3-h6 smaller
Paragraph	`<p>Some text.</p>`	Block of text
Bold	`<strong>text</strong> / <b>`	Important text / visual bold only
Italic	`<em>text</em> / <i>`	Emphasis / visual italic only
Underline / strike	`<u>text</u> / <s>text</s>`	Underline / strikethrough
Highlight / small	`<mark>text</mark> / <small>`	Highlighted / fine print
Super / subscript	`x<sup>2</sup> H<sub>2</sub>O`	Exponents, chemical formulas
Line break / divider	`  <hr>`	New line / horizontal rule
Link	`<a href="https://example.com">Link</a>`	Clickable hyperlink
Image	`<img src="pic.jpg" alt="Description">`	Embeds an image
Lists	`<ul><li>Item</li></ul> <ol>...</ol>`	Bulleted / numbered list
Quote	`<blockquote>Quote</blockquote>`	Indented quotation
Code	`<code>x = 1</code> <pre>...</pre>`	Monospace / keeps whitespace
Containers	`<div>...</div> <span>...</span>`	Group a block / wrap inline text

4. Button → GET request → table

Pick a GET endpoint that returns a list of rows (add sample data if the table is empty). Add the button, an empty output area, and a link to your script at the end of `<body>`:

```
<button id="loadBtn">Load Table</button>
<div id="output"></div>
<script src="/static/app.js"></script>
```

In `static/app.js`: listen for the click, `fetch` the data, build table HTML.

```
document.getElementById("loadBtn").addEventListener("click", async () => {
  const output = document.getElementById("output");
  try {
    const response = await fetch("YOUR API ENDPOINT HERE");   // GET by default
    const rows = await response.json();                       // array of objects
    const columns = Object.keys(rows[0]);                     // headers from 1st row

    let html = "<table border='1'><tr>";
    columns.forEach(col => html += `<th>${col}</th>`);
    html += "</tr>";
    rows.forEach(row => {
      html += "<tr>";
      columns.forEach(col => html += `<td>${row[col]}</td>`);
      html += "</tr>";
    });
    output.innerHTML = html + "</table>";
  } catch (err) {
    output.textContent = "Error: " + err.message;
  }
});
```

5. Form → POST request

Each input needs a **name** that matches a field your POST endpoint expects. Pair every input with a `<label for=...>` pointing at its `id`.

```
<form id="contactForm">
  <label for="name">Name</label>
  <input type="text" id="name" name="name" required>

  <label for="topic">Topic</label>
  <select id="topic" name="topic">
    <option value="general">General</option>
    <option value="support">Support</option>
  </select>

  <label for="message-text">Message</label>
  <textarea id="message-text" name="message" rows="5"></textarea>

  <button type="submit">Submit</button>             <!-- fires the submit event -->
  <button type="reset">Clear</button>               <!-- empties the form -->
  <button type="button" id="cancelBtn">Cancel</button> <!-- does nothing until JS says so -->
</form>
<p id="message"></p>
```

Input type	HTML	Use for
Text	`<input type="text">`	Short text
Email	`<input type="email">`	Email (browser checks the format)
Dropdown	`<select><option>...</option></select>`	Pick one from a list
Text area	`<textarea rows="5"></textarea>`	Long, multi-line text

In `app.js`: stop the page reload, turn the form into JSON, and send it.

```
const API_URL = 'YOUR ENDPOINT HERE';
const form = document.getElementById('contactForm');
const message = document.getElementById('message');

form.addEventListener('submit', async (event) => {
  event.preventDefault();                                  // stop the page reload
  const data = Object.fromEntries(new FormData(form));     // { name, topic, message }
  try {
    const response = await fetch(API_URL, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(data)
    });
    if (!response.ok) throw new Error('Server responded with ' + response.status);
    message.textContent = 'Thanks, ' + data.name + '! Submitted.';
    form.reset();
  } catch (error) {
    message.textContent = 'Something went wrong: ' + error.message;
  }
});

document.getElementById('cancelBtn').addEventListener('click', () => {
  form.reset();
  message.textContent = '';
});
```

The full tutorial version also disables the Submit button and shows "Sending..." while the request runs, then re-enables it in a **finally** block.

6. Switching to another page

Add `templates/view2.html` and a route for it. A form button (or a link) sends the browser there.

```
# app.py
@app.route("/view2", methods=["GET", "POST"])
def view2():
    return render_template("view2.html")
```

```
<!-- index.html: button that opens view 2 -->
<form action="{{ url_for('view2') }}" method="post">
  <button type="submit">Go to View 2</button>
</form>

<!-- view2.html: link back home -->
<a href="{{ url_for('home') }}">Back to home</a>
```

Watch out: `url_for()` takes the Python **function name**, not the URL or file name. If your home route is `def home():`, use `url_for('home')`. Using `url_for('index')` there gives a BuildError.

Key ideas at a glance

Concept	Code	What it does
Render a page	`render_template('page.html')`	Sends an HTML file from templates/
Build a URL	`{{ url_for('func_name') }}`	Flask fills in the route's address
Load a script	`<script src="/static/app.js">`	Runs JS from the static/ folder
Find an element	`document.getElementById("id")`	Grabs an element by its id
React to a click	`el.addEventListener("click", fn)`	Runs fn when the event happens
Call the API	`await fetch(url, options)`	GET by default; set method for POST
Read JSON	`await response.json()`	Turns the reply into JS objects
Show results	`el.innerHTML = html`	Replaces what's inside an element